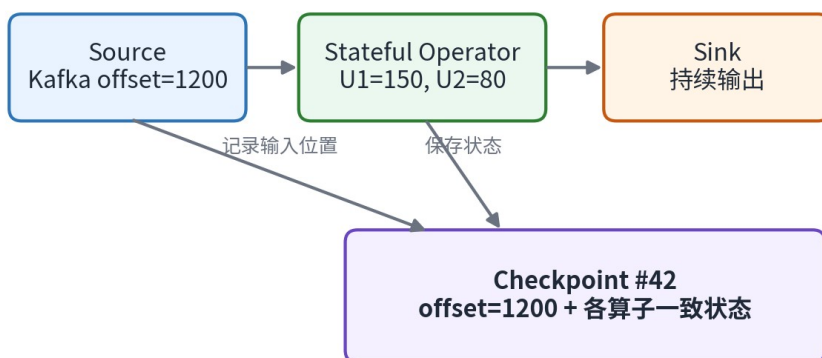


Flink 原理详解：第 6 讲

Checkpoint、Barrier 与 Exactly-once：流不停，怎样得到一致快照？

本讲目标：理解 Flink 容错不是“把内存偶尔保存一下”这么简单，而是要同时解决状态一致性、输入位置、并行算子同步、故障重放和外部输出一致性。

Checkpoint：把“运行中的记忆 + 输入位置”做成可恢复的一致快照



故障后：回到同一个 checkpoint，同时恢复状态和输入位置，再从那里继续。

图 1 Checkpoint 同时保存算子状态与可恢复的输入位置

1. 为什么仅有 State 还不够？

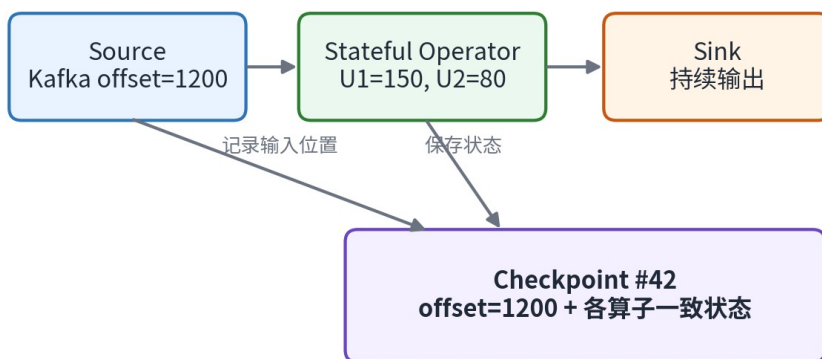
上一讲已经知道：State 让 Flink 能“记住过去”。但内存里的状态会随着 TaskManager 故障一起消失。要让这份记忆在故障之后还能恢复，就必须周期性地“把某一时刻的全局一致状态”持久化下来。这个机制就是 Checkpoint。

一句话：State 解决“运行时记忆”；Checkpoint 解决“这份记忆故障后还能恢复”。

困难在于：Flink 是一个并行、持续运行的数据流系统。每个算子都在不停处理事件，各个 Subtask 的进度还不同。你不能随便在某个瞬间把各台机器的内存各拷一份，因为这些状态可能来自不同的逻辑时间。

2. Checkpoint 到底保存什么？

Checkpoint：把“运行中的记忆 + 输入位置”做成可恢复的一致快照



故障后：回到同一个 checkpoint，同时恢复状态和输入位置，再从那里继续。

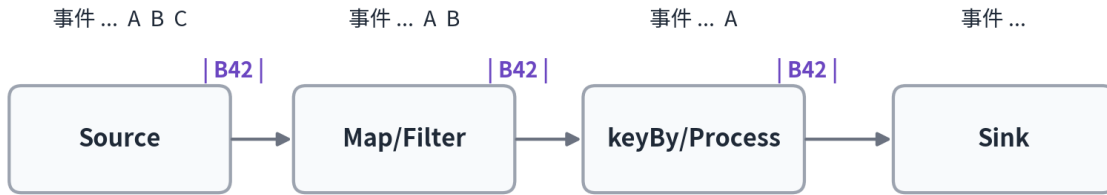
图 2 恢复需要的是“状态 + 输入进度”的一致组合

组成	为什么需要
Operator/Keyed State	恢复累计值、窗口、去重集合、风控状态等
Source 读取位置	例如 Kafka partition offset；恢复时知道从哪里重放
必要的元数据	描述状态属于哪个 Operator/Subtask、如何恢复
某些模式下的 in-flight 数据	Unaligned Checkpoint 会把网络缓冲中的在途数据纳入快照

因此，Checkpoint 不是“把所有输入数据复制一遍”。原始事件仍然保存在可回放 Source 中；Checkpoint 重点保存的是状态与恢复位置。

3. Barrier : Checkpoint 的逻辑分界线

Barrier : Checkpoint 的“分界线”，跟着数据流穿过整个 Job



Barrier #42 前面的事件属于 checkpoint #42；Barrier 后面的事件属于下一段执行。

Barrier 本身不是业务数据，它是控制标记。

图 3 Barrier 是穿过 Dataflow 的控制标记

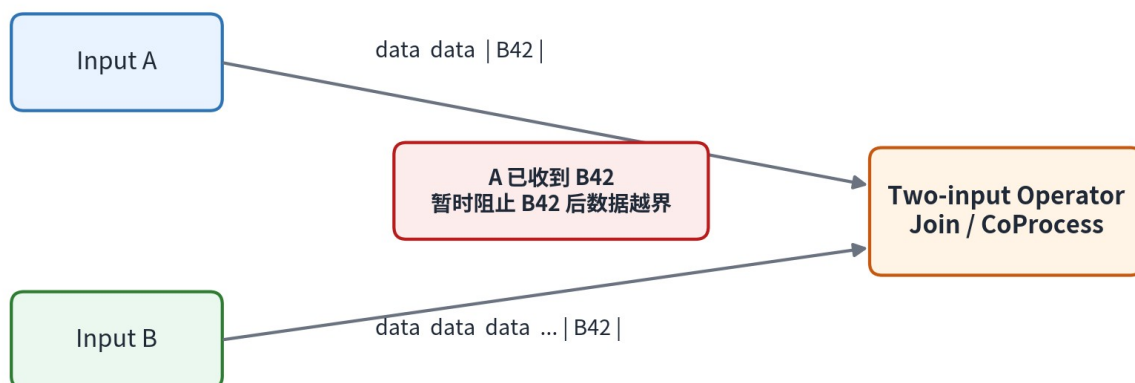
Checkpoint Coordinator 触发某次 checkpoint 后，Source 会先记录自己的读取位置，并向数据流中插入带编号的 Checkpoint Barrier，例如 Barrier #42。Barrier 会像一条“逻辑分界线”沿着 JobGraph 向下游传播。

Barrier #42 的语义不是“现在机器时间是 42”，而是：这个 Barrier 之前的输入属于 checkpoint #42 的一致历史，之后的输入属于下一段执行。

算子收到 Barrier 时，会对自己的受托管状态做快照，并继续把 Barrier 向下游传递。最终，当整个 Job 的相关算子都成功完成快照后，Checkpoint #42 才算完成。

4. 为什么双输入算子需要 Barrier Alignment ？

双输入算子的 Barrier Alignment ：先到的通道要“等一等”



当所有输入都收到同一个 Barrier #42 后，算子状态才对应“同一个逻辑时刻”，然后继续处理。

图 4 两个输入通道必须形成同一个一致切面

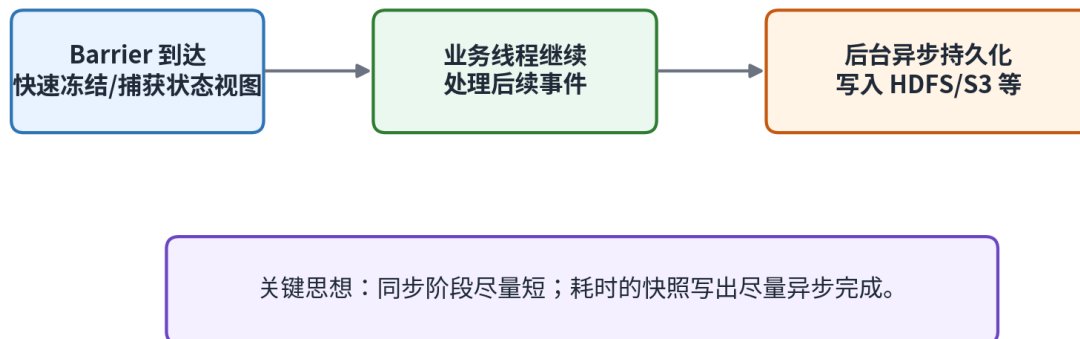
单输入算子比较直观，但 Join、CoProcess 等算子会同时从多个输入通道接收数据。如果 Input A 已经看到 Barrier #42，而 Input B 还没有，说明 A 已经进入“#42 之后”，B 还停留在“#42 之前”。

在经典 aligned checkpoint 中，先收到 Barrier 的通道会暂时阻止 Barrier 后面的记录继续越过这个一致性边界，等待其他输入也收到相同编号的 Barrier。所有输入对齐以后，算子才能得到一个真正一致的状态快照。

底层思想类似数据库的一致快照：不是要求所有机器同一纳秒停住，而是构造一个逻辑上自洽的“切面”。

5. Checkpoint 会不会让整个流处理停下来？

异步快照：Checkpoint 不要求业务计算长期停下来



因此“Checkpoint = 把整个 Job 暂停几秒钟再拷贝内存”是错误理解。

图 5 快照的持久化尽可能异步完成

不会简单地“暂停整个 Job，复制完所有内存，再继续”。Flink 的状态快照机制强调异步：Barrier 到达时，算子首先快速建立一个可快照的状态视图，然后业务处理可以继续，耗时的持久化工作由后台异步完成。

对于大状态，State Backend 还可能使用增量 checkpoint、copy-on-write 或其他机制减少停顿与重复写入。具体实现依赖所使用的状态后端。

6. 故障后到底怎么恢复？

故障恢复：状态与 Source 位置必须一起回退到同一个 Checkpoint



1201~1260 可能会被重新读取和重新执行

“重新处理”不等于“结果重复”：Exactly-once 讨论的是状态/输出最终是否只受影响一次。

图 6 恢复的核心是“状态回退 + Source 回退 + 重新执行”

1. 某个 Task / TaskManager 发生故障，当前执行结果不再可信。
2. Flink 选择最近一次成功完成的 Checkpoint，例如 #42。
3. 各算子从 #42 恢复自己的受托管状态。
4. Source 恢复到 #42 对应的读取位置，例如 Kafka offset=1200。
5. 1201 之后的事件重新读取并重新执行。
6. 最终系统重新回到与无故障执行等价的一致状态。

重要：Exactly-once 不等于“每条消息在 CPU 上只运行一次”。故障后事件完全可能被重新执行；保证的是最终状态语义不会因为重放而多算一次。

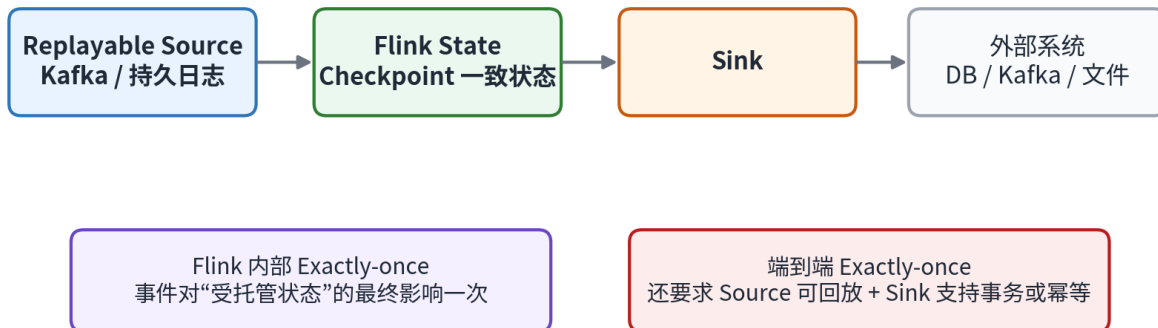
7. At-most-once、At-least-once、Exactly-once

语义	故障后可能怎样	直觉
At-most-once	可能丢，但通常不重复	宁可少算，也不重放
At-least-once	不丢，但可能重复影响结果	重放优先保证不丢
Exactly-once	最终受托管状态等价于每个事件影响一次	允许物理重放，但逻辑结果一致

Flink 的 checkpointing mode 中，EXACTLY_ONCE 是核心模式。AT_LEAST_ONCE 可以放宽对齐要求以换取某些极低延迟场景的性能。

8. 端到端 Exactly-once：为什么 sink 也必须配合？

Exactly-once：先区分“Flink 内部状态一致性”与“端到端结果一致性”



因此：开启 Checkpoint ≠ 自动保证所有外部数据库写入都不重复。

图 7 Flink 内部一致性 $\neq$ 任意外部系统天然 Exactly-once

假设 Flink 正确地恢复了内部 State，但故障前已经向某个普通数据库 INSERT 了一条记录，恢复后这条事件又被重放并再次 INSERT。如果 Sink 没有事务或幂等设计，外部数据库仍然会出现重复。

因此端到端 Exactly-once 通常需要两个条件：

- Source 必须可回放到 checkpoint 记录的位置。
- Sink 必须具备事务提交、两阶段提交语义，或者业务上可幂等写入。

开启 checkpoint 只能让 Flink 的受托管状态具备一致恢复能力；“外部世界是否重复”取决于 connector / sink 的语义。

9. Aligned 与 Unaligned Checkpoint

Aligned vs Unaligned Checkpoint：背压时的两种策略



不是“Unaligned 永远更快”：如果状态存储 I/O 才是瓶颈，它反而可能更重。

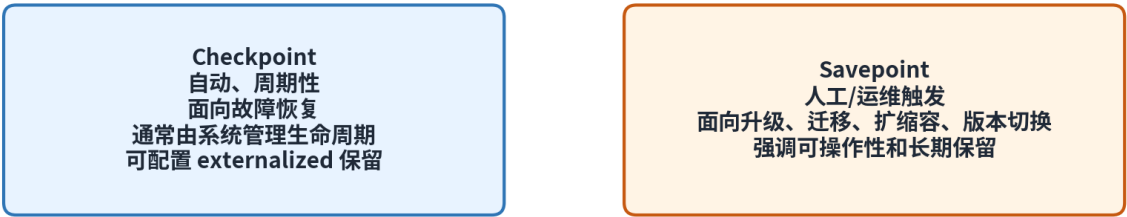
图 8 背压严重时，Barrier 对齐本身可能成为 Checkpoint 延迟来源

正常情况下，aligned checkpoint 很自然：等多个输入的 Barrier 对齐，再建立一致快照。但严重 Backpressure 时，大量网络缓冲会让 Barrier 很久到不了下游，alignment time / start delay 会显著增加。Unaligned Checkpoint 的思路是：不要强求 Barrier 等缓冲完全排空，而是把网络中尚未处理完的 in-flight 数据也作为 checkpoint 的一部分保存。这样 Barrier 可以更快推进，checkpoint 时长对背压更不敏感。

维度	Aligned	Unaligned
一致性	Exactly-once 可用	Exactly-once 可用
背压时 Barrier	可能等待较久	可把在途数据一起快照
Checkpoint 大小	通常较小	可能更大
额外 I/O	通常较少	可能增加状态存储 I/O
适合场景	一般默认思路	Barrier 传播被严重背压拖慢时

10. Checkpoint 与 Savepoint 不要混淆

Checkpoint 与 Savepoint：都能恢复，但目标不同



心智模型：Checkpoint 更像“自动故障保险”；Savepoint 更像“运维迁移存档”。

图 9 Checkpoint 更偏自动容错，Savepoint 更偏运维操作

维度	Checkpoint	Savepoint
主要目标	自动故障恢复	升级、迁移、扩缩容、版本切换
触发	周期性自动触发为主	人工/运维触发
生命周期	通常由系统管理，可配置外部保留	通常长期保留直到人工删除
设计取向	恢复效率	操作灵活性与可迁移性

最直观的比喻：Checkpoint 像“自故障保”；Savepoint 像“你主动保存的迁移存档”。

11. Checkpoint 配置：先理解，不要死背参数

下面是一份概念性 PyFlink 配置示例。不同 Flink/PyFlink 版本的导入路径和可用方法可能略有变化，但配置思想稳定：

```
from pyflink.datastream import StreamExecutionEnvironment, CheckpointingMode

env = StreamExecutionEnvironment.get_execution_environment()

# 每 60 秒触发一次 checkpoint
env.enable_checkpointing(60_000)

config = env.get_checkpoint_config()
config.set_checkpointing_mode(CheckpointingMode.EXACTLY_ONCE)
config.set_checkpoint_timeout(10 * 60_000)
config.set_min_pause_between_checkpoints(30_000)
config.set_max_concurrent_checkpoints(1)
```

生产环境还应把 checkpoint 写入真正的持久化存储，例如 HDFS/S3，而不是只依赖 JobManager 内存。典型配置思想如下：

```
state.backend.type: rocksdb
execution.checkpointing.storage: filesystem
execution.checkpointing.dir: hdfs:///flink/checkpoints
```

与你上一讲衔接：State Backend 决定“运行中的状态怎么管理”；Checkpoint Storage 决定“恢复快照持久化到哪里”。

12. 一完整案例：累

假设 Kafka 中不断产生订单，Flink 按 city keyBy 后累计城市销售额。Checkpoint #42 时：

```
Kafka offsets: p0=1200, p1=980
State: Beijing=125000, Shanghai=83000
Checkpoint #42 -> HDFS
```

随后系统继续处理：北京又增加 5000，上海又增加 2000；但 TaskManager 崩溃。内存中的新状态丢失。恢复时：

7. 从 HDFS 读取 Checkpoint #42 的 State。
8. Kafka p0 回到 1200，p1 回到 980。
9. 之后事件重新读取。
10. 北京/上海重新计算后再次得到故障前应有的正确状态。

如果 Sink 是支持 Exactly-once 的 Kafka connector 或事务性 Sink，可把外部输出也纳入一致提交协议；如果只是普通非幂等 HTTP POST，则不能仅凭 Flink Checkpoint 就宣称端到端 Exactly-once。

13. 为什么 Checkpoint 会变慢？如何观察？

现象	可能原因	观察方向
Checkpoint duration 很长	状态大 / 存储 I/O 慢 / Barrier 传播慢	Checkpoint UI、duration、size
Alignment time 很长	背压严重，多个输入 Barrier 难对齐	alignment / start delay / backpressure
Checkpoint size 持续增长	State 不断膨胀	State size、TTL、业务 key 数
频繁超时失败	间隔过短 / 存储慢 / 资源不足	timeout、concurrent checkpoints
恢复很慢	快照大 / 后端或远端存储吞吐低	restore duration、state backend

调优原则和 Spark 一样：先用指标确认瓶颈，再决定是否调间隔、State、Backend、存储、Backpressure 或 Unaligned Checkpoint。

14. 常见误区

误区	正确理解
Checkpoint 就是定时保存 JVM 内存	它是跨算子的全局一致状态快照机制
Checkpoint 时整个 Job 完全暂停	持久化尽可能异步；对齐只影响特定数据通道
Exactly-once = 每条事件物理执行一次	故障后事件可能重放，保证的是最终语义一致
开了 Checkpoint，数据库就不会重复	端到端语义还取决于 Source/Sink
Barrier 是业务数据	Barrier 是控制标记
Barrier 越频繁越好	过于频繁会增加快照与存储压力
Savepoint 就是手动 Checkpoint 的别名	用途和设计取向不同
Unaligned 永远比 aligned 高级	它主要解决背压下 Barrier 传播问题，也会增加 I/O/快照量

15. 手工推演实验：你自己模拟一个 Checkpoint

不安装 Flink 也可以先用下面的思维实验理解恢复：

```
# checkpoint #7 时
offset = 4
state = {'U1': 150, 'U2': 80}

# checkpoint 后又处理了 3 条
(U1, +20)
(U2, +30)
(U1, +10)

# 然后故障
# 恢复时 state 回到 checkpoint #7, offset 也回到 4
# 再重放后 3 条, 最终仍应得到:
# U1=180, U2=110
```

请注意：后 3 条事件确实“又执行了一遍”，但只要恢复点一致，最终受托管状态与无故障执行相同。

16. 练习题

11. 为什么只保存 State，而不保存 Kafka offset，不能保证一致恢复？
12. 双输入 Join 中，为什么一个输入收到 Barrier 后不能无条件继续处理 Barrier 后数据？
13. Exactly-once 为什么不等于“每条消息只被 CPU 执行一次”？
14. 为什么普通非幂等 HTTP Sink 可能破坏端到端 Exactly-once？
15. 严重背压时，aligned checkpoint 为什么可能变慢？
16. Checkpoint 和 Savepoint 分适合什景？

17. 参考答案

17. 因为恢复后还必须知道 Source 从哪里继续读；只有状态没有输入位置，会导致少读或重复读且无法形成一致切面。
18. 因为两个输入会处在不同逻辑时间；必须形成同一 checkpoint 边界，才能得到一致状态。
19. 故障恢复会从 checkpoint 位置重放事件；Exactly-once 指最终状态语义，而非物理执行次数。
20. 恢复重放时可能再次调用 HTTP；若外部系统不支持事务/幂等，就可能产生重复副作用。
21. 背压导致网络缓冲堆积，Barrier 传播和对齐时间增加。
22. Checkpoint 主要用于自故障恢；Savepoint 主要用于人工升级、迁移、扩缩容等运维操作。

18. 本最心智模型

Source 记录位置并注入 Barrier → Barrier 穿过 Dataflow → 各算子在同一逻辑边界做状态快照 → 快照异步持久化 → Checkpoint 完成。故障后：恢复 State + 恢复 Source 位置 + 重放后续数据。

把这一讲压缩成四句话：

- Barrier 负责在持续流动的数据里画出“一致性边界”。
- Checkpoint 把这个边界上的状态和输入位置持久化。
- Restore 从同一个边界同时恢复状态与输入，再通过重放继续计算。
- Exactly-once 是一致性语义；端到端 Exactly-once 还要求 Source 和 Sink 配合。

19. 下一讲预告：故障恢复之后，系统为什么不会越跑越慢？

下一讲进入 Flink 第 7 讲：Backpressure、网络缓冲与流控。你会看到：当 Sink 处理不过来时，压力为什么会沿着 Dataflow 一层层向上游传播；为什么这不是简单的“线程堵住了”；以及 Backpressure 与 Checkpoint Barrier、吞吐、延迟之间是什么关系。

参考资料

本讲机制核对来源：Apache Flink Stable Documentation - Fault Tolerance、Checkpointing、State Backends、Checkpointing under Backpressure。文档内容以当前 stable 分支为准。